

GUÍA RÁPIDA EXAMEN — CRIPTOGRAFÍA

Referencia de código para examen. Todo está revisado y funciona. Archivos completos en `Criptografia/Examen-Enero/`.

CONCEPTO HÍBRIDO (leer si hay dudas)

Problema: RSA no puede cifrar mensajes grandes. Fernet es rápido pero requiere compartir la clave.

Solución híbrida: 1. El emisor genera una **clave Fernet aleatoria de sesión** (solo para este mensaje). 2. Cifra el mensaje con esa clave Fernet (rápido, sin límite de tamaño). 3. Cifra la clave Fernet con la **clave pública RSA/ECC del receptor** (pequeña, segura). 4. Envía los dos paquetes: mensaje cifrado + clave de sesión cifrada. 5. El receptor usa su **clave privada RSA/ECC** para recuperar la clave Fernet. 6. Con la clave Fernet descifra el mensaje.

EMISOR	RECEPTOR
-- genera clave_fernet	
-- Fernet(clave_fernet).encrypt(msg) -->	msg_cifrado
-- RSA_pub_receptor.encrypt(clave_fernet)	clave_fernet_cifrada
---- paquete viaja por la red ---->	
	RSA_priv_receptor.decrypt(clave_fernet_cifrada) --> clave_fernet
	Fernet(clave_fernet).decrypt(msg_cifrado) --> mensaje original

Firma va aparte: el emisor firma el mensaje en claro con su **privada** antes de cifrar. El receptor verifica con la **pública del emisor** después de descifrar.

1. SIMÉTRICO (Fernet)

Archivo completo: `Criptografia/Examen-Enero/Ejercicio-Simetrico.py`

```
from cryptography.fernet import Fernet

# Generar y guardar clave
clave = Fernet.generate_key()
with open("clave_AB.key", "wb") as f:
    f.write(clave)

# Cargar clave
with open("clave_AB.key", "rb") as f:
    clave = f.read()

fernet = Fernet(clave)

# Cifrar
msg_cifrado = fernet.encrypt(b"Mensaje secreto")

# Descifrar
```

```
msg_original = fernet.decrypt(msg_cifrado)
print(msg_original.decode())
```

Clave: A y B comparten la misma clave. B y C comparten otra. No hay firma.

2. ASIMÉTRICO RSA (cifrar + firmar)

Archivo completo: Criptografia/Examen-Enero/Ejercicio Asimetrico.py

```
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature

# --- GENERAR ---
priv = rsa.generate_private_key(public_exponent=65537, key_size=2048,
    ↪ backend=default_backend())
pub = priv.public_key()

# --- GUARDAR ---
with open("privada_A.pem", "wb") as f:
    f.write(priv.private_bytes(serialization.Encoding.PEM,
        serialization.PrivateFormat.TraditionalOpenSSL, serialization.NoEncryption()))
with open("publica_A.pem", "wb") as f:
    f.write(pub.public_bytes(serialization.Encoding.PEM,
    ↪ serialization.PublicFormat.SubjectPublicKeyInfo))

# --- CARGAR ---
with open("privada_A.pem", "rb") as f:
    priv = serialization.load_pem_private_key(f.read(), password=None)
with open("publica_A.pem", "rb") as f:
    pub = serialization.load_pem_public_key(f.read())

# --- FIRMAR (privada del EMISOR) ---
firma = priv.sign(
    mensaje,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

# --- CIFRAR (pública del RECEPTOR) ---
msg_cifrado = pub.encrypt(
    mensaje,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(),
    ↪ label=None)
)

# --- DESCIFRAR (privada del RECEPTOR) ---
msg = priv.decrypt(
    msg_cifrado,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(),
    ↪ label=None)
```

```

)

# --- VERIFICAR FIRMA (pública del EMISOR) ---
try:
    pub_emisor.verify(firma, msg,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        ↪ salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256())
    print("Firma OK")
except InvalidSignature:
    print("Firma INVALIDA")

```

3. ASIMÉTRICO ECIES (cifrar + firmar con curva elíptica)

Archivo completo: Criptografia/Examen-Enero/Ejercicio-Asimetrico-ECIES.py

```

import hashlib
from ecies.utils import generate_eth_key
from ecies import encrypt, decrypt
from eth_keys import keys

# --- GENERAR ---
priv_key = generate_eth_key()
priv_hex = priv_key.to_hex()
pub_hex = priv_key.public_key.to_hex()

# --- GUARDAR/CARGAR (TXT, no PEM) ---
with open("ecies_claves_A.txt", "w") as f:
    f.write(f"{priv_hex}\n{pub_hex}\n")

with open("ecies_claves_A.txt", "r") as f:
    lineas = f.read().splitlines()
    priv_hex, pub_hex = lineas[0], lineas[1]

# --- CIFRAR (pub_hex del receptor) ---
msg_cifrado = encrypt(pub_hex_receptor, mensaje)

# --- DESCIFRAR (priv_hex del receptor) ---
msg = decrypt(priv_hex_receptor, msg_cifrado)

# --- FIRMAR (priv_hex del emisor, requiere hash SHA256) ---
msg_hash = hashlib.sha256(mensaje).digest()
priv_obj = keys.PrivateKey(bytes.fromhex(priv_hex.replace('0x', '')))
firma_bytes = priv_obj.sign_msg_hash(msg_hash).to_bytes()

# --- VERIFICAR (pub_hex del emisor) ---
pub_obj = keys.PublicKey(bytes.fromhex(pub_hex.replace('0x', '')))
firma_obj = keys.Signature(firma_bytes)
if pub_obj.verify_msg_hash(hashlib.sha256(mensaje).digest(), firma_obj):
    print("Firma OK")

```

```
else:
    print("Firma INVALIDA")
```

Diferencia clave vs RSA: claves en hex (no PEM), funciones encrypt/decrypt directas de ecies.

4. HASH (integridad)

Archivo completo: Criptografia/Examen-Enero/Ejercicio-Hash.py

```
import hashlib
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import hashes

hash_esperado = "55d78a834e7c..." # Dado por el enunciado

# --- HASHLIB (más simple) ---
h = hashlib.sha256()
h.update(segmento_1)
h.update(segmento_2)
h.update(segmento_3)
resultado = h.hexdigest()
print("OK" if resultado == hash_esperado else "CORRUPTO")

# --- CRYPTOGRAPHY ---
digest = hashes.Hash(hashes.SHA256(), backend=default_backend())
digest.update(segmento_1)
digest.update(segmento_2)
digest.update(segmento_3)
resultado = digest.finalize().hex()
print("OK" if resultado == hash_esperado else "CORRUPTO")
```

Nota: finalize() devuelve bytes, hay que llamar .hex() para comparar con string.

5. FIRMA / AUTENTICACIÓN (solo firma, sin cifrar)

```
# RSA - Firmar
firma = priv.sign(
    mensaje,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

# RSA - Verificar
try:
    pub.verify(firma, mensaje,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
    ↪ salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256())
```

```

    print("Autenticado")
except InvalidSignature:
    print("NO autenticado")

# ECIES/ECDSA - Firmar
msg_hash = hashlib.sha256(mensaje).digest()
firma_bytes =
    ↪ keys.PrivateKey(bytes.fromhex(priv_hex.replace('0x', ''))).sign_msg_hash(msg_hash).to_bytes()

# ECIES/ECDSA - Verificar
pub_obj = keys.PublicKey(bytes.fromhex(pub_hex.replace('0x', '')))
es_valida = pub_obj.verify_msg_hash(hashlib.sha256(mensaje).digest(),
    ↪ keys.Signature(firma_bytes))

```

6. HÍBRIDO RSA + Fernet (MAS PROBABLE EN EXAMEN)

Archivo completo: Criptografia/Examen-Enero/Ejercicio-Hibrido.py

```

from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.asymmetric import rsa, padding as asym_padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.fernet import Fernet
from cryptography.exceptions import InvalidSignature

# ---- CIFRADO HÍBRIDO (emisor) ----
def cifrar_hibrido(mensaje, pub_key_destino):
    clave_fernet = Fernet.generate_key()
    msg_cifrado = Fernet(clave_fernet).encrypt(mensaje)
    clave_cifrada = pub_key_destino.encrypt(
        clave_fernet,
        asym_padding.OAEP(mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(), label=None)
    )
    return clave_cifrada, msg_cifrado # <- enviar ambos

# ---- DESCIFRADO HÍBRIDO (receptor) ----
def descifrar_hibrido(clave_cifrada, msg_cifrado, priv_key_destino):
    clave_fernet = priv_key_destino.decrypt(
        clave_cifrada,
        asym_padding.OAEP(mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(), label=None)
    )
    return Fernet(clave_fernet).decrypt(msg_cifrado)

# ---- FLUJO COMPLETO A -> B ----
mensaje = b"Documento confidencial"

# A firma en claro
firma = priv_a.sign(
    mensaje,

```

```

    asym_padding.PSS(mgf=asym_padding.MGF1(hashes.SHA256()),
    ↪ salt_length=asym_padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
# A cifra para B
clave_cifrada, msg_cifrado = cifrar_hibrido(mensaje, pub_b)

# B descifra
msg_recuperado = descifrar_hibrido(clave_cifrada, msg_cifrado, priv_b)

# B verifica firma de A
try:
    pub_a.verify(firma, msg_recuperado,
        asym_padding.PSS(mgf=asym_padding.MGF1(hashes.SHA256()),
    ↪ salt_length=asym_padding.PSS.MAX_LENGTH),
        hashes.SHA256())
    print("Firma de A verificada")
except InvalidSignature:
    print("ERROR: firma invalida")

```

7. HÍBRIDO ECIES + Fernet

Archivo completo: Criptografia/Examen-Enero/Ejercicio-Hibrido-ECIES.py

```

import hashlib
from ecies.utils import generate_eth_key
from ecies import encrypt, decrypt
from cryptography.fernet import Fernet
from eth_keys import keys

# ---- CIFRADO HÍBRIDO (emisor) ----
def cifrar_hibrido(mensaje, pub_hex_destino):
    clave_fernet = Fernet.generate_key()
    msg_cifrado = Fernet(clave_fernet).encrypt(mensaje)
    clave_cifrada = encrypt(pub_hex_destino, clave_fernet) # ECIES cifra la clave
    return clave_cifrada, msg_cifrado

# ---- DESCIFRADO HÍBRIDO (receptor) ----
def descifrar_hibrido(clave_cifrada, msg_cifrado, priv_hex_destino):
    clave_fernet = decrypt(priv_hex_destino, clave_cifrada) # ECIES descifra la clave
    return Fernet(clave_fernet).decrypt(msg_cifrado)

# ---- FLUJO COMPLETO A -> B ----
mensaje = b"Documento confidencial"

# A firma (ECDSA)
msg_hash = hashlib.sha256(mensaje).digest()
priv_a_obj = keys.PrivateKey(bytes.fromhex(priv_hex_a.replace('0x', '')))
firma_bytes = priv_a_obj.sign_msg_hash(msg_hash).to_bytes()

```

```

# A cifra para B
clave_cifrada, msg_cifrado = cifrar_hibrido(mensaje, pub_hex_b)

# B descifra
msg_recuperado = descifrar_hibrido(clave_cifrada, msg_cifrado, priv_hex_b)

# B verifica firma de A (ECDSA)
pub_a_obj = keys.PublicKey(bytes.fromhex(pub_hex_a.replace('0x', '')))
firma_obj = keys.Signature(firma_bytes)
es_valida = pub_a_obj.verify_msg_hash(hashlib.sha256(msg_recuperado).digest(),
    ↪ firma_obj)
print("Firma OK" if es_valida else "Firma INVALIDA")

```

8. ERRORES COMUNES Y FIXES

Error	Causa	Fix
<code>TypeError: a bytes-like object is required</code>	Pasar <code>str</code> en vez de <code>bytes</code>	Añadir <code>.encode()</code> o usar <code>b"..."</code>
<code>cryptography.fernet.InvalidToken</code>	Clave Fernet incorrecta o mensaje corrupto	Verificar que se usa la misma clave para cifrar y descifrar
<code>ValueError: Invalid message (ECIES)</code>	<code>decrypt</code> con clave privada incorrecta	Comprobar que <code>priv_hex</code> y <code>pub_hex</code> son del mismo par
<code>InvalidSignature</code>	Firma verificada con clave pública incorrecta, o mensaje alterado	Usar siempre la pública del emisor para verificar
<code>AttributeError: 'bytes' object has no attribute 'hex' (hashlib)</code>	Usar <code>digest()</code> en vez de <code>hexdigest()</code>	Cambiar a <code>h.hexdigest()</code> o llamar <code>.hex()</code> sobre el resultado
<code>digest.finalize()</code> ya llamado	<code>finalize()</code> solo puede llamarse una vez	Crear nuevo objeto <code>hashes.Hash(...)</code> si necesitas reusar
OAEP: mensaje demasiado largo	RSA solo soporta mensajes pequeños (~190 bytes con 2048 bits)	Usar cifrado HÍBRIDO para mensajes grandes
<code>FileNotFoundError</code> al cargar claves	Las claves no se han generado/guardado antes	Ejecutar primero el bloque de generación y guardado

9. FIRMA: TRES MÉTODOS COMPARADOS

Hash e integridad no son lo mismo que firma. El hash comprueba que el contenido no cambió; la firma prueba además **quién** lo envió (clave privada del emisor).

9.1 RSA — hash interno (lo más habitual en examen)

La librería cryptography aplica SHA256 al mensaje por dentro. Pasas el **mensaje en claro** a `.sign()` y `.verify()`.

```
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

# FIRMAR (privada del emisor)
firma = priv.sign(
    mensaje,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

# VERIFICAR (publica del emisor)
try:
    pub.verify(
        firma, mensaje,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        ↪ salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("Firma OK")
except InvalidSignature:
    print("Firma INVALIDA")
```

9.2 RSA — Prehashed (hash manual explícito)

Equivalente conceptual a ECIES: tú calculas `SHA256(mensaje)` y la firma opera sobre esos **32 bytes**, no sobre el mensaje original.

```
import hashlib
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric.utils import Prehashed
from cryptography.exceptions import InvalidSignature

msg_hash = hashlib.sha256(mensaje).digest() # bytes, 32 – obligatorio

firma = priv.sign(
    msg_hash,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    Prehashed(hashes.SHA256())
)

try:
    pub.verify(
        firma, msg_hash,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        ↪ salt_length=padding.PSS.MAX_LENGTH),
        Prehashed(hashes.SHA256())
    )
```



```

    )
    print("Firma OK (Prehashed)")
except InvalidSignature:
    print("Firma INVALIDA")

```

Importante: con Prehashed, tanto `.sign()` como `.verify()` reciben `msg_hash` (bytes), **no** mensaje.

9.3 ECIES / eth_keys — hash siempre manual

No existe Prehashed en `eth_keys`: siempre firmas el digest de 32 bytes.

```

import hashlib
from eth_keys import keys

msg_hash = hashlib.sha256(mensaje).digest()  # bytes, 32

priv_obj = keys.PrivateKey(bytes.fromhex(priv_hex.replace('0x', '')))
firma_bytes = priv_obj.sign_msg_hash(msg_hash).to_bytes()

pub_obj = keys.PublicKey(bytes.fromhex(pub_hex.replace('0x', '')))
firma_obj = keys.Signature(firma_bytes)
valida = pub_obj.verify_msg_hash(msg_hash, firma_obj)  # True / False
print("Firma OK" if valida else "Firma INVALIDA")

```

9.4 HMAC-SHA256 — autenticación simétrica (no es firma asimétrica)

Usa la **misma clave compartida** del canal. Da integridad + autenticidad entre quienes conocen la clave, pero **no** prueba identidad ante un tercero (cualquiera con la clave puede generar el MAC).

```

import hmac
import hashlib

# Generar MAC (emisor y receptor comparten clave_compartida)
mac = hmac.new(clave_compartida, mensaje, hashlib.sha256).digest()  # bytes

# Verificar (usar compare_digest, no == directo)
valido = hmac.compare_digest(mac, mac_recibido)
print("MAC OK" if valido else "MAC INVALIDO")

```

Método	Clave	Entrada a firmar	Uso típico
RSA hash interno	Par RSA	mensaje (bytes)	Examen híbrido RSA
RSA Prehashed	Par RSA	<code>sha256(m).digest()</code>	Cuando el enunciado pide “firmar el hash”
eth_keys ECDSA	Par ECIES hex	<code>sha256(m).digest()</code>	Examen híbrido ECIES
HMAC	Clave simétrica compartida	mensaje (bytes)	Canal Fernet sin RSA (ej. ejercicio 4 evolucionado)

10. BYTES, HEX Y BASE64: CUÁNDO USAR CADA UNO

Modos de apertura de ficheros

Modo	Qué lee/escribe	Usar para
"rb" / "wb"	bytes puros	Claves .pem, .key, mensajes cifrados, medicos.txt cifrado
"r" / "w"	str Unicode	Claves ECIES .txt (hex), usuarios.txt en texto plano
"ab"	añadir bytes al final	Registrar usuarios sin borrar el fichero (CreTxt.py)

Error típico: `for linea in f.read()` en modo "rb" itera **enteros** (0-255), no líneas. Correcto: `mode="r"` y `for linea in f:`.

Hash: digest vs hexdigest

```
import hashlib

h = hashlib.sha256(mensaje)

digest_bytes = h.digest()      # bytes, 32 → comparar, firmar, HMAC
hex_str      = h.hexdigest()   # str, 64 chars → JSON, prints, ficheros legibles

# Conversiones
bytes.fromhex("a4b90387...")  # hex str → bytes
digest_bytes.hex()            # bytes → hex str
```

Base64 (empaquetar binario en JSON)

```
import base64

b64_str = base64.b64encode(datos_bytes).decode()  # str para JSON
origen  = base64.b64decode(b64_str)              # bytes originales
```

Regla general

- **Operar en Python** (cifrar, descifrar, firmar, comparar): `bytes (.digest(), b"...", .encode())`.
- **Guardar en JSON o mostrar:** mensaje y firma en **base64**; hash en **hex** (`hexdigest()`).
- **Ficheros de usuarios** (`medicos.txt`): texto con , separador; salt y hash en base64 como **str** dentro de cada línea.

Tabla de conversiones rápidas

De	A	Cómo
bytes	hex str	<code>b.hex()</code>
hex str	bytes	<code>bytes.fromhex(s)</code>
bytes	base64 str	<code>base64.b64encode(b).decode()</code>

De	A	Cómo
base64 str	bytes	base64.b64decode(s)
str	bytes	s.encode()
bytes	str	b.decode()

Formato del hash según contenedor

Situación	Formato	Ejemplo
Comparar en Python (==)	<code>.digest()</code> → bytes	<code>sha256(m).digest() == hash_guardado</code>
JSON / texto legible	<code>.hexdigest()</code> → str hex	<code>payload["hash_msg"] = sha256(m).hexdigest()</code>
Firmar con <code>eth_keys</code>	<code>.digest()</code> → bytes (obligatorio)	<code>priv.sign_msg_hash(sha256(m).digest())</code>
Firmar RSA Prehashed	<code>.digest()</code> → bytes (obligatorio)	<code>priv.sign(msg_hash, ..., Prehashed(...))</code>
Payload JSON híbrido	mensaje base64, hash hex, firma base64	Ver sección 7 y ejercicio 3

RESUMEN DE REGLAS CLAVE

CIFRAR → usar clave PÚBLICA del RECEPTOR
 DESCIFRAR → usar clave PRIVADA del RECEPTOR
 FIRMAR → usar clave PRIVADA del EMISOR
 VERIFICAR → usar clave PÚBLICA del EMISOR

RSA → claves en .pem | OAEP (cifrar) y PSS (firmar)
 ECIES → claves en .txt hex | `encrypt(pub_hex, msg)` / `decrypt(priv_hex, msg)`
 ECDSA → firmar siempre `sha256(mensaje).digest()` con `eth_keys`
 HMAC → `hmac.new(clave_compartida, mensaje, sha256).digest()` – solo canal simétrico